Reproducing Repeated Padding for Sequential Recommendation: Replication and Extension to GitHub Star Sequences

Pitipat Tanangkorn (48611794)
Macquarie University
pitipat.tanangkorn@students.mq.edu.au

Abstract

This report reproduces **Repeated Padding (RepPad)**, a data augmentation method for sequential recommendation published at RecSys 2024 (Dang et al., 2024). RepPad fills the empty padding space of short user sequences with repeated copies of the user's own history instead of zeros. We first check whether the original claim holds with the authors' official code, then test it on new data, including a new dataset of GitHub repository-starring sequences that we constructed. With the official code, RepPad improved SASRec clearly on Beauty (HR@10 +21.5% over 5 seeds), Toys (+11.6%, 3 seeds) and LastFM (+12–13%), in the same direction as the paper but with different sizes. On our GitHub data it was *worse* than the baseline by 11–19% (10 and 5 seeds), and a blind LLM-as-judge found no difference in relevance between the two systems' lists. All code and scripts are available in our public repository.[1]

1 Introduction

Sequential recommenders such as GRU4Rec and SASRec predict the next item a user will interact with from their past interactions. Our project follows a two-stage design: (i) replicate the source work using its original code, data and evaluation framework; (ii) construct new data and re-evaluate the core claim of the paper on it. Section 2 describes the paper, Section 3 its datasets, Section 4 our replication, Section 5 the new data, Section 6 the results on it, Section 7 ethical considerations and Section 8 our conclusions.

2 Description of the Paper and Justification

What the paper proposes. Neural sequential models need inputs of a fixed maximum length N, so short histories are padded with zeros. These positions carry no information, yet most users have short histories because user behaviour is long-tailed (Dang et al., 2024). RepPad instead repeats the user's own sequence s_u in the padding space, separated by a delimiter: $s_u^p = [s_u \,|\, 0 \,|\, s_u \,|\, \cdots \,|\, 0 \,|\, s_u]$. For example, with $N = 8$ the history $[i_1, i_2, i_3]$ becomes $[0, i_1, i_2, i_3, 0, i_1, i_2, i_3]$ rather than $[i_1, i_2, i_3, 0, 0, 0, 0, 0]$. It adds no trainable parameters and no extra training sequences, so it is a pure pre-processing step. The authors' explanation is that short-sequence users contribute too few training targets compared with long-sequence users, and repetition rebalances this; they also report faster loss convergence and more concentrated item-embedding gradients.

How it is evaluated. The paper uses Hit Rate (HR@K) and NDCG@K with $K \in \{5, 10\}$, under leave-one-out evaluation after 5-core filtering: the last item of each user is the test item, the second-last is for validation, and the rest is for training. RepPad is compared with plain zero-padding on the same backbone (GRU4Rec and SASRec) and with other augmentation methods such as CMRSI. The reported gains reach 60.3% for GRU4Rec and 24.3% for SASRec on short-sequence datasets, with little or no benefit on the long-sequence ML-1M dataset.

Why this paper. RepPad appeared at the 18th ACM Conference on Recommender Systems (RecSys 2024), which CORE ranks A (CORE, 2023) after being promoted from B in 2021 (RecommenderSystems.com, 2021). The idea is simple, has few hyperparameters, and the authors release code for both backbones,[2] so any difference between our results and the paper is easier to investigate. It has also attracted follow-up work (Dang et al., 2024b, 2026, 2024c).

[1] https://github.com/ppeakyplus-hub/reppad

[2] https://github.com/KingGugu/RepPad

Dataset	#Users	#Items	Avg. Len
Toys	19,412	11,924	8.6
Beauty	22,363	12,101	8.9
Sports	35,958†	18,357	8.3
Yelp	30,431	20,033	10.4
Home	66,519	28,237	8.3
ML-1M	6,040	3,706	165.6

Table 1: Original dataset statistics reported by Dang et al. (2024). †The data file in the official repository has 35,598 users, so the paper's 35,958 is probably a typo.

3 Description of the Original Datasets

The paper evaluates on six public datasets: four Amazon review categories (McAuley et al., 2015) (Toys, Beauty, Sports, Home), Yelp, and MovieLens-1M (Harper and Konstan, 2015).[3] Table 1 gives the statistics reported in the paper. The Amazon and Yelp datasets are short-sequence (average length 8–10), whereas ML-1M is long (165.6). (The abstract says "five datasets", but Section 3.2 and Table 3 of the paper list six, and the main results tables label five of them as short-sequence datasets.) The official code also ships LastFM, which we use for our first replication runs.

The official code reads one text file per dataset, with one line per user: the user id followed by that user's item ids in time order (ids start at 1; 0 is the padding value). For example, the first lines of `Beauty.txt` are:

```
1 1 2 3 4 5
2 6 7 8 9 10 4 11
3 4 12 13 14 15 16 17 18 19
```

so user 2 interacted with items 6, 7, 8, 9, 10, 4 and 11, in that order. The last item of a line is the test item, the one before it the validation item, and the rest is the training sequence (leave-one-out).

4 Replication of the Original Work

4.1 Environment

We ran the official implementation (SASRec backbone) with `--aug_type=0` (zero-padding baseline) and `--aug_type=1` (RepPad). All runs used one MacBook Pro (Mac16,8) with an Apple M4 Pro chip (12 cores: 8 performance, 4 efficiency), 24 GB of memory, macOS, Python 3.13.9 and PyTorch 2.13.0, on the CPU only (no CUDA GPU was used). We did not use a university VM. One dependency, `gensim`, is missing from the repository's requirements and had to be installed manually.

[3]Amazon: https://jmcauley.ucsd.edu/data/amazon/; Yelp: https://www.yelp.com/dataset; MovieLens-1M: https://grouplens.org/datasets/movielens/1m/. We used the already processed files shipped with the official code and did not download the raw data.

Metric	Baseline	RepPad	Gain
HR@5	.0471 ± .0044	**.0521** ± .0040	+10.5%
NDCG@5	.0328 ± .0022	**.0357** ± .0024	+9.0%
HR@10	.0669 ± .0033	**.0747** ± .0041	+11.8%
NDCG@10	.0391 ± .0011	**.0431** ± .0023	+10.1%

Table 2: SASRec on LastFM with the official code, mean ± std over 10 seeds.

4.2 Results

Table 2 shows our results on LastFM (CPU only, about 100 seconds per run), as mean ± standard deviation over 10 random seeds (0–9) for each setting. RepPad is better on average on all four metrics. Our first single run suggested gains of 19–37%, but this was an optimistic seed: the 10-seed mean gain is about 9–12%. RepPad was better than the baseline in 7–10 of the 10 seeds depending on the metric, and a paired t-test over seeds gives $p < 0.05$ for all four metrics in this first set of runs. With only 10 seeds and four correlated metrics we treat this as consistent evidence of a gain, not as a precise estimate. LastFM is not one of the paper's six datasets, so there is no reported number to compare with; it only shows that the direction agrees with the paper's claim.

Re-running seeds 0–4 later (to save per-user predictions, Section 6) gave different numbers for the same seeds, so training is not fully deterministic. In those five re-runs the gains at $K{=}10$ held (HR@10 +12.7%, NDCG@10 +7.6%), but the gains at $K{=}5$ almost disappeared (HR@5 +0.3%, NDCG@5 +1.1%). We therefore regard the LastFM gain as real at $K{=}10$ and uncertain at $K{=}5$.

Beauty: comparison with the paper. On Beauty (5 seeds, about 33 minutes per run on CPU) RepPad improves clearly over the baseline, in line with the paper (Table 3). A paired t-test over seeds gives $p < 0.001$ for all four metrics (RepPad is higher in every seed). Our baseline is within 0.003 of the paper's, and our RepPad numbers are close to or slightly above the paper's (HR@10 .0745 vs. .0726; NDCG@10 .0411 vs. .0390). Our relative gains (+21.5% HR@10, +28.0% NDCG@10) are larger than the paper's (+14.0%, +17.8%), mostly because our baseline is slightly lower. We cannot

	HR@10		NDCG@10	
	Base	RepPad	Base	RepPad
Ours (5 seeds)	.0613	.0745	.0321	.0411
Paper	.0637	.0726	.0331	.0390

Table 3: Beauty: ours (mean over 5 seeds, std $\leq$ .0014) vs. the numbers in the paper.

	HR@10		NDCG@10	
	Base	RepPad	Base	RepPad
Ours (3 seeds)	.0738	.0824	.0419	.0472
Paper	.0716	.0843	.0396	.0476

Table 4: Toys and Games: ours (mean over 3 seeds) vs. the paper.

say why from the paper alone; possible reasons are the random seed, training details such as early stopping, and small preprocessing differences (metrics are computed over the full item ranking, so test-time negative sampling is not a factor). We therefore claim that the *direction and size order* of the effect replicate, and not the exact numbers.

Toys and Games. On Toys (3 seeds, about 30 minutes per run) RepPad again helps in every seed: HR@10 .0738 $\rightarrow$ .0824 (+11.6%) and NDCG@10 .0419 $\rightarrow$ .0472 (+12.7%) (Table 4). The gain is smaller than the paper's (+17.7% and +20.2%): our baseline is higher than the paper's (.0738 vs. .0716 HR@10) while our RepPad HR@10 is a little lower (.0824 vs. .0843). With only three seeds the p-values ($p < 0.01$ for all metrics) are weak evidence, and we report this dataset as consistent in direction but not in size.

Where RepPad helps and where it does not. RepPad helps on LastFM, Beauty and Toys, and hurts on our GitHub data (Section 6). We did not run ML-1M or the other datasets (Sports, Home, Yelp) for lack of time on CPU-only hardware, so the paper's long-sequence case is covered only by LastFM, which is not one of its datasets. The differences from the paper's numbers could come from seeds, early stopping and preprocessing details; we did not run experiments to separate these.

5 Construction of New Data

5.1 Existing datasets not used in the paper

The paper reports six datasets (Table 1). The official repository also ships **LastFM**, which the paper does not report, so it is an existing dataset that was not used in the paper. We use it as our existing extra dataset. It is already in the same (user, item, order) format as the other files, so we did not convert it ourselves; we computed its statistics from the file (Table 5). We did not verify the original source or the filtering that was applied to it, and we say so as a limitation. Compared with the paper's Amazon data it has far longer sequences (48.2 on average, up to 899), which makes it a useful contrast. We could not download further public datasets (for example another Amazon category) from the environment where we worked, so we do not claim more.

Dataset	#Users	#Items	#Inter.	Avg. Len	Sparsity
LastFM (existing)	1,090	3,646	52,551	48.2	0.987
GitHub (ours)	1,421	972	31,783	22.4	0.977

Table 5: Datasets outside the paper, after 5-core filtering (statistics computed from the files used in our runs).

5.2 GitHub repository-starring sequences

Source and collection. We built a new dataset from the public GitHub REST API with a three-step script (included in the repository). A first attempt that picked users by follower count failed: the 106 users we collected starred 3,834 different repositories and only 7 of them were starred by five or more of these users, so 5-core filtering removed everything. Items must be shared between users, so the final design fixes the item vocabulary first. (1) The item vocabulary is the 1,000 most-starred repositories (Search API, the maximum it returns). (2) Users are sampled with the Search Users API (`type:user repos:>=10 followers:>=1`) restricted to one randomly drawn day of account creation between 2012 and 2024, taking up to 20 users per day, so the sample contains ordinary developers from different cohorts rather than famous accounts. (Sampling from repository stargazers was our first choice, but the stargazers endpoint returned 404 in our setup.) (3) For each user we call `GET /users/{username}/starred` with the `application/vnd.github.star+json` media type, which returns the `starred_at` timestamp, read up to three pages of 100 stars (the most recent 300), and keep only stars on repositories in the vocabulary. Users with fewer than five such stars are dropped. Sorting by timestamp gives (user, item, timestamp) sequences in the same format as the original datasets. Requests are authenticated and the script waits for the rate-limit reset when the 5,000 requests/hour quota is nearly used.

Final dataset. We sampled 3,000 users; 1,427 had at least five stars on the 1,000 vocabulary repositories (31,877 interactions over 999 repositories, average length 22.3). After 5-core filtering the dataset has 1,421 users, 972 items and 31,783 interactions (average sequence length 22.4, sparsity 0.977), so almost no user is lost and the sequences are short, which is the regime where RepPad is claimed to help (Table 5).

Limitations. Items are restricted to 1,000 popular repositories, so the data describes popular-repository starring, not all starring behaviour. Only the 300 most recent stars of each user are read, which truncates long histories. Users come from the Search API (accounts with at least 10 repositories and one follower), so inactive and very new accounts are under-represented. Stars of a repository are not equivalent to consumption or purchase: users may star to bookmark, so the sequences are noisier than purchase data. These choices keep sequences short (22.4 on average, none above 123 after filtering), which on paper is the regime where RepPad should help; it did not (Section 6).

Snippet. The collection script writes a CSV with the columns `user,item,timestamp`, where `user` is a salted hash and `timestamp` is the time of the star in UTC. The conversion script turns it into the RepPad text format below (user id, then the repository ids in time order, shortened here) and a separate item map. We show only indexed rows and not rows of the raw CSV, since these would pair a pseudonym with real timestamps.

```
GitHub.txt
1 1 2 3 4 5 6 7
3 47 48 19 49 50

GitHub_item_map.csv (excerpt)
item_id,repository
2,ionic-team/ionic-framework
3,freeCodeCamp/freeCodeCamp
```

6 Results on New Data

6.1 GitHub repository-starring data

Table 6 gives the mean and standard deviation over 10 seeds (SASRec, same settings as Section 4). On this dataset RepPad is *worse* than the baseline on all four metrics, by 11–14%. A paired t-test over seeds gives $p \leq 0.005$ for every metric (RepPad is lower in 9 of 10 seeds for HR@5 and NDCG@5 and in all 10 for HR@10 and NDCG@10), and all four remain significant after a Bonferroni correction for four metrics ($0.05/4 = 0.0125$). The four metrics are correlated, and 10 seeds is a small sample, so we treat this as consistent evidence of a negative effect on this dataset and not as a precise estimate.

	Baseline	RepPad
HR@5	.0804±.0080	.0693±.0040
NDCG@5	.0500±.0051	.0442±.0016
HR@10	.1324±.0045	.1161±.0071
NDCG@10	.0667±.0038	.0592±.0023

Table 6: GitHub data, mean±std over 10 seeds (SASRec).

History length	Users	HR@10 base	RepPad	Change
5–10	452	.1292	.1146	−11.3%
11–24	491	.1430	.1141	−20.2%
25–123	478	.1444	.1096	−24.1%

Table 7: GitHub: HR@10 by history length (mean over 5 re-runs).

Repeatability. We re-ran seeds 0–4 later with the same code and seeds to save per-user predictions, and the numbers were not identical (for example, seed-0 baseline HR@10 changed from .1372 to .1302), so the training is not fully deterministic even with a fixed seed. The mean over these five re-runs gives a larger drop (about −18%) than the original 10 seeds (−11 to −14%). We therefore trust the *direction* of the GitHub result, which is the same in both sets, more than its exact size, and the standard deviations in Table 6 do not include this extra variation.

Short versus long histories on GitHub. We split the 1,421 users into three groups of similar size by history length and used the five re-runs (Table 7). RepPad is below the baseline in every group, so the claim that it helps short sequences is not supported on this dataset. The loss is smallest for the shortest histories (HR@10 −11.3%) and largest for the longest (−24.1%), which fits the paper's reasoning in direction (repeating helps more when there is more padding to fill) but not in sign.

Short versus long histories on LastFM. The same split on LastFM (1,090 users, five re-runs, Table 8) shows the opposite pattern from GitHub: RepPad is above the baseline in all three groups, and the gain is *largest* for the longest histories (HR@10 +23.2%) and smallest for the shortest (+8.5%). Together, the two datasets do not support a simple rule that RepPad helps mainly short sequences. On GitHub the relative gain is best for short users, on LastFM it is best for long users, and the sign differs. Each group is only 342–492 users and five runs, so the per-group differences are indicative and we did not test them.

History length	Users	HR@10 base	RepPad	Change
5–10	342	.0959	.1041	+8.5%
11–34	378	.0656	.0730	+11.3%
35–899	370	.0443	.0546	+23.2%

Table 8: LastFM: HR@10 by history length (mean over 5 re-runs).

Dataset	Avg. len	Seeds	HR@10 change
Toys (paper)	8.6	3	+11.6%
Beauty (paper)	8.9	5	+21.5%
GitHub (new)	22.4	10 / 5	−12.3% / −18.9%
LastFM (ext.)	48.2	10 / 5	+11.8% / +12.7%

Table 9: RepPad vs. baseline across datasets (HR@10, relative change). Two values are the first set of seeds and the later re-runs.

Possible explanations (hypotheses, not tested). We did not run experiments that isolate the cause, so the following are hypotheses. (i) Order may carry little signal in GitHub stars: people star to bookmark, and many popular repositories are starred in waves, so the next star is hard to predict from the order and repeating the sequence adds noise. In LastFM the histories are listening logs with strong order. (ii) The vocabulary is only 972 popular repositories, so the baseline already predicts well from popularity, leaving little room for an augmentation. (iii) Very long histories are cut at the model's maximum length, so repetition has nothing to fill for them.

Does the short-vs-long claim hold? Table 9 puts the datasets side by side. The paper's claim is that RepPad helps short sequences most. Our results agree for the two Amazon datasets (short, helped) but not as a general rule: LastFM has the longest histories and is also helped, and GitHub, which is in between, is hurt. Average length alone therefore does not predict the sign of the effect in our data, and within LastFM and GitHub the gains by length group point in opposite directions. With four datasets we cannot say more than this.

6.2 LLM-as-judge evaluation

Goal and data. HR and NDCG only count whether the one held-out repository is in the top-10. We therefore asked an LLM whether the *recommended lists* are relevant to a user's interests. This is possible only for the GitHub data, because it is the only dataset where items have readable names (the LastFM and Beauty files contain numeric ids). The judge saw the user's last 15 starred repositories before the validation and test items, and two lists of ten repository names, with no information about which system produced which.

Open-source judge: failed validation. Following the proposal, we first used an open-source model with a fixed version (Qwen2.5-7B through Ollama, digest `845dbda0ea48`, temperature 0). A 1–5 rating of each list gave scores of 1–2 for almost everyone, and a per-repository yes/no rating gave about 1% "relevant" items. After a pilot with 10 users we changed the prompt twice (finer-grained judgements, then a pairwise A/B comparison in both orders to cancel position bias); this was a change made after seeing pilot results, so we report it as such. We then ran a *validity check* that does not involve any recommender: for 50 users with long histories, the judge chose between their 10 real later stars and 10 random repositories. It chose the real ones 50.0% of the time, with "tie or inconsistent between the two orders" in all 50 cases, so this judge cannot tell relevant from irrelevant lists with names alone, and we did not use it to evaluate RepPad.

Blind judging by Claude. As a fallback we used Claude (the assistant that also helped us write code and text for this project, so it is *not* an independent judge) on a blind sample of 60 cases in random order and with randomised A/B positions: 30 validity-check cases (real later stars vs. random repositories) and 30 cases comparing the baseline's and RepPad's top-10 for the same user (seed 0). The key was generated by our script and withheld until the judge had answered. On the validity cases the judge picked the real stars in 27 of 28 non-tied cases (96.4%; sign test $p < 0.001$), so it separates relevant from irrelevant lists. On the model cases RepPad was preferred 3 times, the baseline 3 times, and the lists were judged equally relevant 24 times: the judge sees *no difference in relevance* between the two systems.

Reading the result. This does not contradict the lower HR/NDCG of RepPad on GitHub, but it limits what that drop means. The two systems often recommend almost the same popular repositories, and the judge, which sees only names, rates both lists as similarly on topic. The loss in HR/NDCG therefore does not show up as less topical recommendations; it is a loss in predicting the exact next star. Limitations: one judge that is not independent, names only (no descriptions), 30 model cases, one seed, no human annotation, and the validity check

used real later stars and random repositories, which is an easy contrast, so it shows that the judge is not broken but not that it can detect small differences. Per-user judgements and the script that builds and scores the blind sample are in the repository.

7 Ethical Considerations

Ethics sections are often a one-line disclaimer: in ACL 2021 only about 16% of the papers had one, and about 20% of those did not address any of the benefits, harms or vulnerable groups that the authors looked for (Benotti and Blackburn, 2022). A large corpus of ACL ethics statements likewise finds that more than a third identify no issue at all, and groups the concerns that do appear into bias, misuse, privacy, misinformation, toxicity and environmental impact (Karamolegkou et al., 2025). Good statements are concrete about data handling, for example whether personal information was collected and how identities were protected (Pyatkin et al., 2023). We follow this approach, using the concern categories above where they apply to a recommender-systems project.

Original paper. RepPad is trained and evaluated on public interaction logs (product reviews, business reviews, movie ratings and music listening histories). These are behavioural records of real people. Although the user identifiers are pseudonymous, sparse interaction histories can sometimes be linked back to individuals, so *privacy* is the main concern. In the arXiv version of the paper that we read we found no ethics statement or ethical-considerations section; its only limitations discussion (Section 2.5.2) is technical. We did not check whether the published ACM version adds one. The second concern is *bias*. Recommenders trained on logs tend to favour users with long histories and popular items. RepPad is interesting here because its mechanism gives short-history users more training signal (Section 2), which may reduce this under-representation. The same mechanism, however, repeats a user's own past behaviour, so it could also reinforce existing preferences and narrow what is recommended; the original evaluation, based on HR and NDCG, does not measure this. Compute-related *environmental impact* is small, since RepPad adds no parameters or training sequences and our runs used a laptop CPU.

Our new data. GitHub stars are public, but they are still behavioural data about identifiable individuals who did not agree to be part of a research dataset. Stars can also reveal interests that are sensitive in some contexts, such as security tools or political projects. Because we sample ordinary developers (not only well-known accounts), they have a higher expectation of privacy than public figures, so we collect only what is needed. Anonymising a starred list is also weaker than it looks, because the same list can be read from the public API by anyone, so replacing usernames with indices does not stop re-identification by matching. We therefore replace usernames with salted hashes at collection time (the raw handle list is deleted when processing finishes and the salt is not published), number users in the released file. The public repository contains the collection script, the item map (public repository names) and a small indexed sample of the sequences, but never usernames or the salt; we do not publish the full user-level file. We also respect the API's authentication and rate limits and collect only the fields needed (user, repository, time). We have not carried out a legal review of GitHub's Terms of Service and acceptable use policies, which govern use of its data; we used only the official REST API with authentication, within its rate limits, and did not scrape web pages. We did not use human annotation, so there are no annotator agreement or payment issues; our LLM-as-judge evaluation raises the separate issue that judge models can carry their own biases.

Responsible conduct of research. The Australian Code for the Responsible Conduct of Research (NHMRC et al., 2018) is built around honesty, rigour, transparency, fairness, respect, recognition, accountability and promotion. For this project this means: reporting limitations openly (single runs, a restricted and popularity-biased sample); releasing code and scripts so the work can be reproduced; and treating the people behind the data with respect. If we were to publish a paper on the GitHub data, we would also need to check whether the work needs ethics review or an exemption as research using public data about people, and we have not asked our institution whether it would require review; the National Statement on Ethical Conduct in Human Research and the university's ethics office are the places to check, and we do not claim that no review is needed.

8 Conclusions and Reflections

Replication. With the official code, RepPad improved SASRec over the zero-padding baseline on Beauty (HR@10 .0613 → .0745, 5 seeds) and Toys (.0738 → .0824, 3 seeds), close to the paper's numbers in size order and identical in direction, although our relative gains differ from the paper's (larger on Beauty, smaller on Toys). We could not run all six datasets, ML-1M in particular, so we cannot confirm the paper's claim that RepPad does little on long sequences. A single run was misleading on LastFM (an apparent gain of 19–37% shrank to about 9–12% over 10 seeds), and training was not deterministic even with fixed seeds, so we report several seeds and trust directions more than exact sizes.

New data. On the GitHub starring data that we constructed, RepPad was worse than the baseline by 11–19% on all four metrics, and a blind LLM judge, which passed its validity check, saw no difference in relevance between the two lists. The paper's hypothesis that RepPad helps short sequences is therefore not supported as a general rule: LastFM (long histories) was helped, GitHub (shorter histories) was hurt, and the gains by history length went in opposite directions on the two. The first hypothesis for this is that GitHub stars carry little order information, but we did not test it.

Reflections. Our first data collection chose users by follower count and failed because items were not shared between users, so 5-core filtering removed everything; fixing the item vocabulary first solved it, at the price of a popularity-biased dataset. A small open-source LLM judge looked usable but failed a simple validity check, which is why we ran this check before using any judge result. With more time we would run ML-1M and the other datasets, repeat the GitHub experiment with item descriptions and a stronger independent judge, and test the hypotheses in Section 6 directly (for example by shuffling the order of stars).

References

Eunjoon Cho, Seth A. Myers, and Jure Leskovec. 2011. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1082–1090.

Luciana Benotti and Patrick Blackburn. 2022. Ethics consideration sections in natural language processing papers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 4509–4516.

CORE Conference Portal. 2023. ACM International Conference on Recommender Systems (RecSys) — Conference Ranking. https://portal.core.edu.au/conf-ranks/28/.

Yizhou Dang, Yuting Liu, Enneng Yang, Guibing Guo, Linying Jiang, Xingwei Wang, and Jianzhe Zhao. 2024. Repeated Padding for Sequential Recommendation. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24)*, pages 497–506.

Yizhou Dang, Yuting Liu, Enneng Yang, Guibing Guo, Linying Jiang, Jianzhe Zhao, and Xingwei Wang. 2024. Repeated Padding+: Simple yet Effective Data Augmentation Plugin for Sequential Recommendation. *arXiv preprint arXiv:2403.06372v3*.

Yizhou Dang, Enneng Yang, Yuting Liu, Guibing Guo, Linying Jiang, Jianzhe Zhao, and Xingwei Wang. 2024. Data Augmentation for Sequential Recommendation: A Survey. *arXiv preprint arXiv:2409.13545*.

Yizhou Dang, Yifan Wu, Minhan Huang, Chuang Zhao, Lianbo Ma, Guibing Guo, Xingwei Wang, and Zhu Sun. 2026. Pay Attention to Sequence Split: Uncovering the Impacts of Sub-Sequence Splitting on Sequential Recommendation Models. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19.

Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 197–206.

Antonia Karamolegkou, Sandrine Schiller Hansen, Ariadni Christopoulou, Filippos Stamatiou, Anne Lauscher, and Anders Søgaard. 2025. Ethical concern identification in NLP: A corpus of ACL Anthology ethics statements. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11618–11635.

Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. 2015. Image-based Recommendations on Styles and Substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 43–52.

National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2018. *Australian Code for the Responsible Conduct of Research*. Commonwealth of Australia, Canberra.

Valentina Pyatkin, Jena D. Hwang, Vivek Srikumar, Ximing Lu, Liwei Jiang, Yejin Choi, and Chandra Bhagavatula. 2023. ClarifyDelphi: Reinforced clarification questions with defeasibility rewards for social and moral situations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11253–11271.

Recommender-Systems.com. 2021. The ACM RecSys Conference becomes an 'A' Conference. https://recommender-systems.com/news/2021/06/08/acm-recsys-conference-a-conference-core/.